



# **Techniques de débogage pour les modèles de Machine Learning et Deep Learning**

# Table des matières

|                                                                                     |           |
|-------------------------------------------------------------------------------------|-----------|
| <b>I - Contexte</b>                                                                 | <b>3</b>  |
| <b>II - Principes et méthodologie de débogage en IA</b>                             | <b>4</b>  |
| A. Les fondamentaux du débogage en IA .....                                         | 4         |
| B. Stratégies de débogage systématique.....                                         | 7         |
| <b>III - Techniques de débogage pour les modèles de Machine Learning</b>            | <b>9</b>  |
| A. Débogage des problèmes liés aux données.....                                     | 9         |
| B. Débogage des algorithmes et des hyperparamètres .....                            | 11        |
| <b>IV - Techniques de débogage pour le Deep Learning</b>                            | <b>13</b> |
| A. Débogage de l'architecture des réseaux de neurones.....                          | 13        |
| B. Débogage du processus d'entraînement .....                                       | 14        |
| <b>V - Synthèse</b>                                                                 | <b>16</b> |
| <b>VI - Auto-évaluation</b>                                                         | <b>20</b> |
| <b>VII - Exercice : Déboguer un modèle de reconnaissance d'images en production</b> | <b>24</b> |

# I Contexte

## Contexte

Déboguer un modèle d'intelligence artificielle n'a rien d'anodin et présente des défis inédits comparé au débogage logiciel classique. Les erreurs y sont rarement explicites : elles se manifestent par des prédictions incohérentes, des pertes soudaines de performance, ou des comportements non reproductibles qui peuvent frustrer même les ingénieurs chevronnés. Dans le débogage IA, la difficulté tient autant à la **nature stochastique des algorithmes** (résultats variables à chaque apprentissage), qu'à l'**interdépendance profonde entre la qualité des données, le code, les paramètres et l'infrastructure**. Une anomalie peut provenir d'un jeu de données mal formaté, d'un bug discret dans le pipeline, d'un mauvais choix d'hyperparamètres, ou d'un problème d'architecture réseau difficile à cerner.

**Le débogage efficace en IA exige une rigueur analytique, une méthode scientifique stricte, mais aussi une certaine intuition développée par l'expérience.** Il ne s'agit pas seulement de « *corriger un bug* », mais de naviguer un espace de recherche immense où chaque étape - de la préparation des données à l'optimisation des architectures - peut contenir des pièges subtils. Cela suppose de **formuler des hypothèses testables, de concevoir des expériences contrôlées, de valider chaque modification par observation rigoureuse**, puis d'itérer méthodiquement jusqu'à la résolution.

Cette leçon explorera ces méthodologies en profondeur :

- Comment organiser sa démarche pour localiser une erreur diffuse,
- Comment s'appuyer sur l'analyse comparative, la simplification progressive et les tests unitaires adaptés à l'IA pour isoler un problème,
- Comment construire un environnement technique propice au débogage (traçabilité, reproductibilité, outils spécialisés, journalisation fine des essais et des métriques).

## II Principes et méthodologie de débogage en IA

### A. Les fondamentaux du débogage en IA

#### Spécificités du débogage des modèles d'IA

Déboguer un modèle d'IA constitue un défi bien plus complexe que le débogage logiciel classique. Contrairement à une application traditionnelle qui s'arrête ou génère un message d'erreur explicite en cas de dysfonctionnement technique, un système d'IA peut rester silencieux alors même qu'il « *déraille* ».

La difficulté principale vient du fait que **les erreurs sont souvent diffuses, statistiquement distribuées ou intermittentes** : il peut s'agir d'une dégradation progressive de la performance sur certains segments, d'une instabilité lors du réentraînement ou d'écarts ponctuels excessifs sur des catégories d'inputs.

La **stochasticité native** de nombreux algorithmes (initialisation aléatoire, batchs différents à chaque époque, augmentation de données, sampling de seeds, etc.) rend difficile la reproductibilité exacte d'un comportement anormal, obligeant à raisonner en termes de tendances plutôt que sur des cas isolés.

De plus, en IA, les problèmes peuvent naître ou se propager à travers toute la chaîne :

- Un défaut subtil de codage dans le prétraitement,
- Un minuscule déséquilibre dans le dataset qui s'amplifie à l'inférence,
- Un choix inadapté d'hyperparamètre qui ne fait surface qu'en fin de parcours,
- Une dépendance logicielle ou matérielle qui introduit du bruit.

Un exemple célèbre est celui d'une équipe projet dans la santé qui a vu son modèle d'imagerie se dégrader après un changement de fournisseur de scanner : l'erreur venait d'un prétraitement de teinte couleur mal adapté, passé inaperçu lors des tests unitaires initiaux.

**La clé est d'intégrer tôt dans chaque projet IA l'idée que chaque couche (données, prétraitement, architecture, code, environnement d'exécution) doit pouvoir être auditée et testée séparément.**

Un bon débogueur en IA doit ainsi déployer une posture d'enquêteur : considérer toute la chaîne, du problème métier jusqu'au code, comme potentielle « *scène de crime* ».

#### Pipeline d'IA

#### Az Définition

Un **pipeline d'IA** désigne la chaîne complète de traitement : collecte et préparation des données → feature engineering → entraînement du modèle → validation → déploiement → inférence. Lors du débogage, chaque maillon peut contenir une erreur (donnée, code, paramètre ou infra), il est donc crucial de pouvoir **tester et isoler chaque étape séparément**.

## Débogage stochastique en IA

Az Définition

On parle de **débogage stochastique** quand les erreurs ou dérives sont liées directement à la dimension aléatoire du processus d'apprentissage (choix des batchs, initialisation, ordre de lecture, augmentation de données, etc.) - elles diffèrent d'une exécution à l'autre. Cela impose de mener des séries d'expériences, de répéter les tests et d'analyser la distribution des résultats plutôt que de chercher une récurrence exacte à chaque essai.

## La science du débogage en IA : entre méthode et intuition

### Le piège du « *tout-code* » dans le débogage IA

⚠ Attention

Le réflexe de « *corriger le code* » peut mener à une impasse si l'on oublie d'explorer la donnée ou les paramètres. Dans de nombreux cas, **l'erreur est structurelle** (dataset, variabilité temporelle, biais de collecte, etc.) et non logicielle. Une « *vision pipeline* » est donc cruciale pour un diagnostic fiable et reproductible.

## Le cycle de débogage itératif

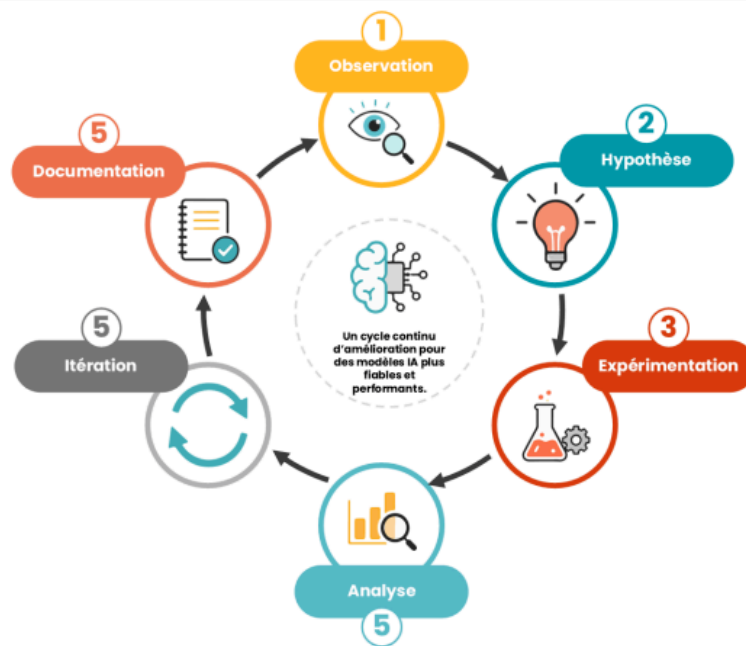
La résolution efficace d'un problème de modèle IA repose sur l'adoption d'un **cycle itératif rigoureux** qui suit la logique scientifique. Ce cycle, utilisé dans les meilleures équipes d'IA en France comme à l'international, s'apparente à de la recherche expérimentale :

- **Observation** : collecter, sans a priori, tous les symptômes et anomalies repérés par les utilisateurs, les métriques ou lors de la validation automatique et les documenter de façon précise (dates, circonstances, types de données concernées, log d'exécution...).
- **Hypothèses** : élaborer plusieurs explications possibles du problème : changement de la distribution des données, bug dans le feature engineering, problème d'architecture, etc.
- **Expérimentation** : modifier un seul paramètre ou composant à la fois (données, code, paramètres, environnement). Créer des expériences contrôlées permet d'isoler le ou les facteurs responsables.
- **Analyse** : mesurer précisément les écarts, interpréter les résultats au regard de l'hypothèse testée, ajuster les métriques de suivi si nécessaire, puis formuler un plan d'action.
- **Itération** : si le problème n'est pas résolu, on recommence le cycle, en affinant ou réorientant les hypothèses en s'appuyant sur les nouveaux indices collectés.
- **Documentation systématique** : garder un historique clair de chaque test et décision prise, de façon à pouvoir revenir en arrière ou partager le diagnostic au sein de l'équipe.

Prenons l'exemple d'une usine qui doit prédire une panne sur ses machines : l'équipe a d'abord observé que les prédictions étaient systématiquement décalées lors des changements de poste (équipe de nuit vs jour). Après divers tests (vérification des capteurs, du prétraitement temporel, du modèle), ils ont fini par découvrir qu'un bug dans la synchronisation des timestamps entre deux machines causait l'erreur.

C'est cette discipline itérative et rigoureuse qui garantit l'efficacité d'un débogage IA, même sur des modèles très complexes.

## Schéma du cycle itératif de débogage IA



### Journalisez vos démarches



Tenir une trace concise de chaque test, des modifications apportées et de l'hypothèse testée permet à la fois de mieux collaborer au sein de l'équipe et de ne pas perdre de vue ce qui a déjà été tenté, surtout sur de longues séquences de debugging.

### La construction d'un environnement de débogage efficace

Le **débogage en IA nécessite un environnement technique solide**. Il faut pouvoir isoler chaque composant du pipeline, garantir la reproductibilité des expériences et tracer chaque modification.

- **Isolation** : tester séparément chaque étape (prétraitement, feature engineering, modèle, post-traitement), c'est s'assurer qu'on localise rapidement l'origine du problème. Par exemple, des données brutes jamais altérées dans une « *sandbox* » aident à détecter si une erreur vient du preprocessing ou a été présente dès la collecte.
- **Reproductibilité** : fixer les seeds aléatoires, figer les versions logicielles et documenter l'état des données garantit que l'on peut répéter n'importe quel test, utile aussi bien pour le travail en équipe que face à des auditeurs externes.
- **Traçabilité et journalisation** : utiliser des notebooks, des outils de suivi d'expériences (Mlflow, DVC) et une gestion de versions claire permet de revenir sur chaque étape et de comprendre la succession des essais et itérations.

Dans l'assurance, une équipe qui devait corriger un modèle de tarification a réussi à localiser une erreur subtile grâce à l'import, sur un serveur isolé, de la version exacte du jeu de données et du code en vigueur lors du bug : **la reproductibilité a permis de rendre un bug invisible (lié au passage à l'heure d'hiver) reproductible**, puis corrigeable.

### Bug d'horodatage en assurance

[👁 Exemple](#)

Une équipe d'assurance a investigué des incohérences de tarification. Grâce à un environnement reproductible, ils ont découvert qu'un **changement silencieux d'heure d'hiver dans les timestamps** faussait l'alignement des données. Sans reproductibilité stricte, ce bug subtil serait resté introuvable.

## B. Stratégies de débogage systématique

### L'approche par simplification progressive

La **simplification progressive** est une stratégie-clé dans le débogage IA : quand le modèle est trop complexe pour qu'on puisse deviner la source du bug, on le réduit étape par étape (moins de variables, moins de couches, version basique du pipeline). Cette réduction peut révéler que le problème disparaît une fois certains composants supprimés - ce qui oriente vers la cause racine.

Exemple concret : lors du développement d'un modèle de reconnaissance de texte manuscrit, l'équipe réduisait graduellement le nombre de couches du réseau. Le bug de surapprentissage chronique ne se produisait que sur les architectures profondes. L'ajout progressif de couches a révélé non pas un problème d'architecture... mais une mauvaise gestion du dropout à partir d'un certain nombre de couches.

Ce type d'approche fonctionne aussi sur l'échantillonnage : commencer avec un tout petit nombre d'exemples, itérer, puis réintroduire la complexité pour identifier le point de bascule du bug ou de la dégradation.

### Ablation (en IA)

[Az Définition](#)

L'ablation consiste à retirer, une à une, des features, des couches ou des modules d'un pipeline ou modèle pour observer leur effet sur la performance ou sur l'apparition d'un bug. Si enlever une composante fait disparaître l'erreur, celle-ci était sans doute en cause ; à l'inverse, si rien ne change, il faut investiguer ailleurs.

### Le débogage par contrastes et comparaisons

Comparer deux versions du modèle, ou deux jeux de données similaires mais générant des résultats opposés, reste l'une des méthodes de diagnostic les plus puissantes. Cela consiste à observer systématiquement :

- Ce qui distingue les cas où le modèle fonctionne de ceux où il échoue,
- Les différences entre deux versions du modèle, surtout après l'introduction d'une nouveauté qui coïncide avec l'apparition d'un bug,
- L'écart de performance d'un pipeline entre plusieurs groupes de clients, d'horaires ou de régions.

Un projet e-commerce a mis au jour un biais régional dans ses recommandations produits uniquement en comparant les échecs d'une version à l'autre, et en cartographiant les causes via des outils d'analyse différentielle.

**Conservez un « *modèle référence* »**

Garder toujours une version stable du modèle comme **baseline de comparaison**. Cela permet de savoir immédiatement si une modification nouvelle est responsable de l'apparition (ou disparition) d'un bug.

**Conservez toujours des jeux de tests fixes pour toute la durée du projet**

Construisez dès le début une « *banque* » d'exemples texts (inputs + résultats attendus pour chaque composant du pipeline). Ces jeux de tests permettent de valider que chaque évolution n'introduit pas de régression cachée, et servent de filet de sécurité pour vérifier rapidement l'efficacité de chaque correctif.

**La stratégie des tests unitaires en IA**

Le test unitaire appliqué à l'IA consiste à découper le pipeline entier en petites briques (lecture, formatage, transformation, encodage, modélisation, inférence, etc.) et à **tester l'intégrité, la cohérence et la stabilité de chaque brique séparément**. Un test unitaire doit réagir à toute modification susceptible d'introduire ou de corriger un bug localisé (ex : modification du mapping d'une variable catégorielle, changement du code d'initialisation, changement de version d'une librairie).

Dans le secteur santé, une équipe a pu corriger un bug insidieux sur le pipeline de preprocessing d'images grâce à un test unitaire qui vérifiait le format couleur d'entrée : le modèle apprenait sur des images dont les canaux étaient inversés, biaisant toute la chaîne.

**Ne négligez jamais la reproductibilité dans le débogage IA**

Si l'environnement n'est pas strictement reproductible (versions, random seed, corpus de tests figé), toute correction d'un bug pourra s'avérer temporaire ou impossible à valider pour l'équipe ou pour un auditeur : sauvegardez, versionnez, et documentez systématiquement chaque état de votre projet.



# III Techniques de débogage pour les modèles de Machine Learning

## A. Débogage des problèmes liés aux données

### Identification et correction des anomalies dans les données

La majorité des bugs en Machine Learning trouvent leur source non pas dans l'algorithme, mais dans les données : **erreurs, valeurs aberrantes, incohérences ou biais** constituent des causes majeures de performances médiocres ou de comportements inattendus du modèle.

Le diagnostic commence donc par une **exploration approfondie des distributions** : visualiser les variables à l'aide d'histogrammes, de boxplots ou de scatter plots met souvent en évidence des anomalies invisibles dans les statistiques globales.

Des techniques automatiques, comme la détection d'**outliers** (valeurs extrêmes) par l'écart interquartile ou la méthode de Mahalanobis, permettent d'isoler les exemples problématiques : l'examen de ces cas peut révéler un bug de collecte, une erreur humaine lors du renseignement, voire une corruption d'un fichier.

En pratique, une entreprise de crédit scoring a identifié une chute soudaine de performance : l'analyse minutieuse des valeurs d'entrée a montré que certaines variables étaient codées différemment selon les agences, générant des catégories incohérentes pour le modèle global. L'étape suivante est la **correction** : soit exclure les données aberrantes, soit les corriger avec des règles métier validées, soit alerter l'équipe responsable pour éviter la récurrence.

#### Données incohérentes en retail

 Exemple

Un site e-commerce avait des milliers de transactions avec des **dates dans le futur**. Ces anomalies ont faussé un modèle de prévision de ventes jusqu'à ce qu'un contrôle automatisé les détecte.

#### Utilisez des scripts automatisés pour les contrôles de cohérence

 Conseil

La plupart des erreurs de données peuvent être évitées grâce à la mise en place de scripts de validation systématiques au sein du pipeline : contrôles de bornes, vérification de l'absence de doublons, check de la logique métier (par exemple, une date de naissance dans le futur doit alerter immédiatement).

## Techniques avancées de validation des données d'entrée

### Débogage des pipelines de prétraitement

Le prétraitement des données (feature engineering, normalisation, encodage, etc.) est une étape critique : une erreur à ce stade compromet tout le modèle. Le diagnostic passe d'abord par une **comparaison rigoureuse des distributions avant et après transformation** : il s'agit de vérifier, pour chaque variable, que les transformations (scaling, log, one-hot encoding, etc.) ont l'effet attendu.

Des erreurs classiques incluent :

- Un mauvais encodage des variables catégorielles (ordre implicite là où il n'en fallait pas, absence de gestion des valeurs inconnues),
- L'application de transformations statistiques sur des données non normalisées,
- L'inversion de colonnes ou la réinjection accidentelle de la target (data leakage).

Un cas typique dans le retail : une société a vu la qualité de prévision de ses ventes s'effondrer après l'ajout d'une transformation logarithmique mal calibrée, qui a « éteint » la variabilité des données sur plusieurs produits.

Un bon débogage impose pour chaque étape critique, de **visualiser côte à côte les distributions** et d'inclure dans les tests unitaires la validation de la cohérence des encodages.

### Les bugs « silencieux » du prétraitement

⚠ Attention

Certains bugs n'occasionnent ni crash ni alerte explicite : une normalisation appliquée deux fois, une colonne renommée sans modification du mapping, un minuscule changement de librairie peuvent introduire des erreurs qui ne se manifestent que par une lente dérive de performance ou un biais subtil mais constant.

### Outlier

Az Définition

Un **outlier** (ou valeur extrême) correspond à une observation très éloignée de la tendance centrale dans une variable. Sa présence peut être un signal de saisie erronée, de rupture de série ou d'événement exceptionnel, et elle doit toujours alerter le data scientist lors de la phase de diagnostic.

## Gestion des déséquilibres de classes

Dans de nombreux contextes, le problème à prédire est rare (fraude bancaire, panne d'équipement, etc.), et le nombre d'exemples positifs est bien plus faible que celui des cas négatifs. Ce **déséquilibre de classes** biaise l'apprentissage : le modèle tend à prédire la classe majoritaire sans s'intéresser vraiment à la classe que l'on cherche à détecter.

Un premier diagnostic consiste donc à **analyser la répartition des classes**, à inspecter les performances par classe (précision, rappel, F1-score pour chaque label), et à visualiser la matrice de confusion : on met ainsi en évidence les « *classes fantômes* » (jamais détectées) ou les surestimations.

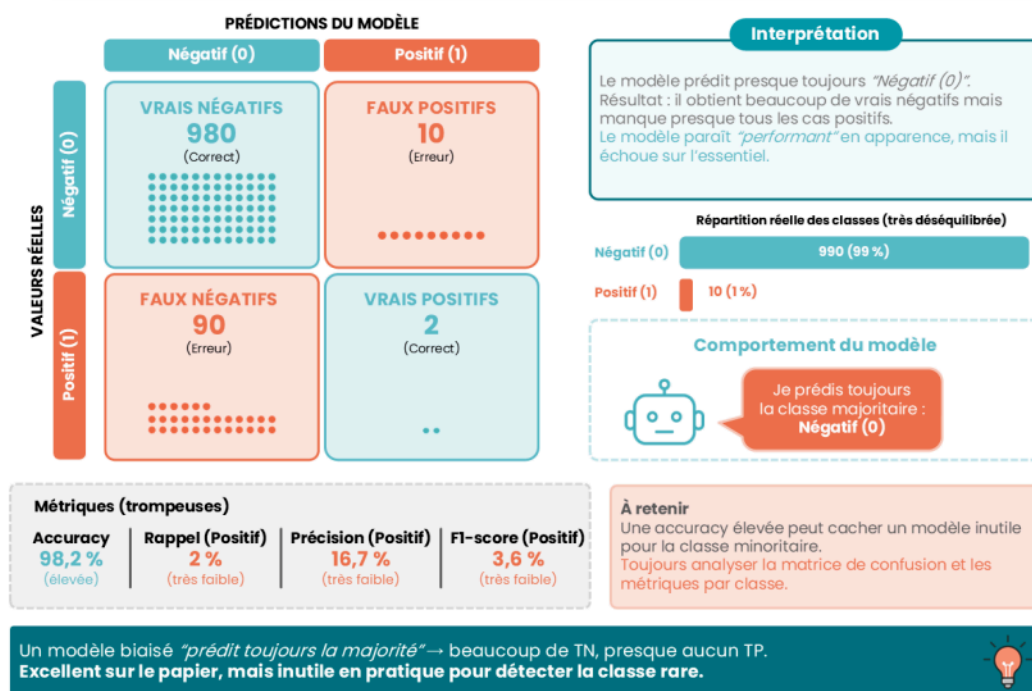
Les solutions pour corriger cette faiblesse sont multiples :

- **Rééchantillonnage** (sur-échantillonnage positif, sous-échantillonnage négatif, SMOTE, etc.),
- **Pondération adaptée** de la fonction de perte,
- **Recours à des** algorithmes spécialisés ou à des modèles d'ensemble,
- Séparation des pipelines de traitement.

Un système de détection de fraude d'une grande banque française a doublé son taux de détection en combinant pondération de la fonction de perte et jeux d'entraînement équilibrés par SMOTE, tout en gardant un test rigoureux de l'impact sur le taux de faux positifs.

## Schéma d'une matrice de confusion déséquilibrée

Beaucoup de vrais négatifs, presque aucun vrai positif.  
Le modèle est biaisé : il **prédit toujours la majorité** (la classe négative).



### Testez l'effet des corrections sur un jeu de test indépendant

Conseil

Après toute action visant à rééquilibrer les classes, il faut valider la robustesse du modèle sur un échantillon indépendant pour éviter le surapprentissage ou la création de nouveaux biais inexpliqués.

## B. Débogage des algorithmes et des hyperparamètres

### Analyse et correction des problèmes de sous-apprentissage

Le sous-apprentissage (underfitting) se manifeste quand le modèle n'arrive pas à capter la structure du problème, donnant de mauvais résultats aussi bien sur le training set que sur la validation. Pour le diagnostiquer, on surveille de près l'évolution des courbes d'apprentissage : si les performances stagnent à un niveau bas et les deux courbes restent proches mais peu élevées, le problème est structurel.

Les principales causes sont :

- Un modèle trop simple ou trop régularisé,
- Un manque de données ou de diversité,
- Des features mal construites ou non informatives.

Pour corriger, il s'agit d'augmenter la complexité du modèle (plus grande profondeur, ajout de features pertinentes, etc.), de revoir la stratégie de régularisation, ou d'acquérir plus de données qualitatives.

### Sous-apprentissage (underfitting)

**Az Définition**

On parle de **sous-apprentissage** lorsqu'un modèle ne parvient pas à apprendre la structure sous-jacente des données, produisant de faibles scores aussi bien en entraînement qu'en validation – le plus souvent à cause d'une capacité trop limitée.

### Résolution des problèmes de convergence

Parfois, l'entraînement du modèle ne parvient pas à optimiser correctement la fonction de perte : la loss oscille, stagne ou retombe sur un plateau. Le débogage consiste alors à :

- Vérifier l'adéquation de la fonction de coût à la nature du problème,
- Ajuster le taux d'apprentissage (learning rate),
- Tester différentes méthodes d'initialisation ou d'optimisation (SGD vs Adam, etc.),
- S'assurer que les données sont bien normalisées,
- Surveiller la consistance des gradients.

Par exemple, dans le scoring client d'une société de service, l'adoption d'un learning rate scheduler adaptatif a permis de sortir d'un minimum local et d'atteindre des performances attendues sans augmenter la taille du jeu de données.

### Stratégies de correction des problèmes d'hyperparamètres

Les hyperparamètres (taux d'apprentissage, nombre de couches/arbres, taille de batch, régularisation, etc.) pilotent directement les capacités d'apprentissage du modèle. Un mauvais choix peut entraîner aussi bien un sous-apprentissage qu'un surapprentissage ou des instabilités. Le débogage passe par des grilles de recherche, le recours à l'optimisation bayésienne, ou des recherches incrémentales (test d'un hyperparamètre après l'autre).

Dans la publicité en ligne, une équipe française a doublé le ROI de ses campagnes en optimisant de façon systématique le taux d'apprentissage et la profondeur de ses arbres via une recherche bayésienne, plutôt qu'aléatoire ou empirique.

### Ne changez qu'un paramètre à la fois pour diagnostiquer

 **Conseil**

Pour isoler l'effet précis de chaque hyperparamètre, modifiez-en un seul par expérience et suivez précisément les changements de performance : cela permet de bâtir des courbes de sensibilité utiles et de retenir les bons réglages sur la durée.

### Validez la robustesse sur plusieurs jeux de test

 **Attention**

Une amélioration obtenue après tuning d'un hyperparamètre doit être vérifiée sur plusieurs splits de données ou jeux indépendants : un gain sur le jeu de validation d'origine peut masquer un surajustement ou un effet artificiel lié à une coïncidence sur l'échantillon.

### Hyperparamètres en publicité en ligne

 **Exemple**

Une régie a découvert que **réduire de moitié le batch size** améliorait significativement le ROI de ses campagnes car le modèle apprenait plus finement les signaux rares.

## IV Techniques de débogage pour le Deep Learning

### A. Débogage de l'architecture des réseaux de neurones

#### Identification et résolution des problèmes de gradient

Dans les réseaux de neurones profonds, une classe d'erreurs fréquentes s'explique par une mauvaise **propagation du gradient** - c'est-à-dire la difficulté, lors de l'apprentissage, d'ajuster correctement les poids de toutes les couches. Deux phénomènes majeurs sont à surveiller : la **disparition** (le gradient devient presque nul et il n'y a plus d'apprentissage dans les couches profondes), et l'**explosion** (le gradient devient énorme, entraînant l'instabilité numérique et la divergence de l'entraînement).

Le diagnostic démarre par le **suivi précis des valeurs de gradient, couche par couche** : des outils de visualisation intégrés dans les frameworks modernes (TensorBoard, Weights & Biases) permettent de détecter les couches « *inactives* » (neurones morts) ou, à l'inverse, des zones où « *tout déraile* ».

Dans un projet de traitement d'images médicales en France, l'équipe a stabilisé un modèle pathologique en adoptant la **batch normalization** et en ajustant précisément l'initialisation des poids.

Une bonne pratique est d'utiliser des méthodes « *anti-gradient vanishing/explosion* » dès la conception : choix de fonctions d'activation adaptées (ReLU, Leaky ReLU, etc.), normalisation des couches, initialisation intelligente (He, Xavier, etc.), analyse itérative des logs pour vérifier la stabilité d'apprentissage.

#### Neurone mort

#### Az Définition

Un **neurone mort** désigne, dans un réseau, une unité qui retourne toujours la même valeur (typiquement zéro) après activation, quelle que soit l'entrée testée. Cela traduit souvent un blocage du gradient ou un mauvais choix de fonction d'activation, et nuit à la capacité du modèle à apprendre des représentations utiles.

#### Débogage des problèmes d'architecture trop complexe ou trop simple

Un **réseau trop complexe** (trop de couches, trop de paramètres, trop de connections) mémorise le bruit, sur-apprend, et décroche sur de nouvelles données ; un réseau trop simple (peu de couches, faible dimensionnalité) n'attrape pas les subtilités, plafonne vite, et stagne sur l'apprentissage.

Pour diagnostiquer :

- On observe la **différence entre erreurs entraînement et validation** (surapprentissage évident ?).
- On analyse la croissance de la capacité du modèle par rapports successifs : si l'ajout de couches n'améliore plus la validation ou la dégrade, c'est souvent un signe de surcapacité.

Dans une entreprise de reconnaissance faciale, le dimensionnement optimal a été obtenu en testant plusieurs architectures et en validant à chaque réduction la stabilité et la robustesse sur des images de profils rares ou bruitées.



**L'illusion du modèle « toujours plus gros »** Attention

Ajouter des couches ou des neurones n'assure pas une meilleure performance : cela augmente la complexité, consomme plus, et accroît le risque de bugs difficilement décelables (défaillance locale d'un chemin, ralentissement caché, etc.).

**Résolution des problèmes spécifiques aux architectures avancées**

Les réseaux récurrents (RNN, LSTM) et transformers posent des défis propres :

- Les **RNN** ont du mal à apprendre ou retenir les dépendances de long terme (problème de « *gradient qui s'efface* » sur les longues séquences).
- Les **Transformers** peuvent représenter des défauts d'attention : certains tokens monopolisent indûment l'attention, ou le modèle ne parvient pas à discerner les dépendances utiles.

L'analyse se mène par :

- Visualisation des poids d'attention et suivi de leur distribution,
- Manipulation de séquences tests extrêmes (phrases longues/complexes, traductions rares, etc.),
- Monitoring des gradients à travers le temps.

Dans la traduction automatique, une entreprise française a détecté un biais de traduction uniquement présent sur des phrases longues grâce à une analyse fine de l'attention : le modèle « *oubliait* » la fin des phrases en anglais lorsque la séquence dépassait 30 tokens.

**Problème d'attention en NLP** Exemple

Un transformer traduisait bien des phrases courtes mais perdait le sens pour des phrases > 30 tokens. La visualisation des poids d'attention a révélé que le modèle « *oubliait* » systématiquement la fin des phrases.

**Préparez des jeux de séquences tests variés et extrêmes** Conseil

Ces jeux de cas limites (phrases très longues, images très bruitées, exemples peu courants) devraient toujours être utilisés lors du débogage des architectures avancées : c'est souvent là que les problèmes se révèlent.

**B. Débogage du processus d'entraînement****Diagnostic et correction des problèmes de batch size et learning rate**

Le choix de la **taille de batch** et du **taux d'apprentissage (learning rate)** influence fortement la vitesse, la stabilité et la qualité finale de l'entraînement. Des symptômes typiques d'un learning rate trop élevé sont des oscillations sauvages, une divergence de la loss ; un batch trop petit apporte du bruit trop important, ralentissant la convergence, tandis qu'un batch trop grand peut aboutir à un défaut de généralisation ou à l'impossibilité de terminer l'apprentissage faute de mémoire.

Le diagnostic s'appuie sur l'observation des **courbes de loss** : oscillations, plateaux, ou perte de performance soudaine à une certaine époque sont des indices. Ajuster graduellement ces paramètres, utiliser le « *learning rate finder* » ou tester des stratégies adaptatives comme le warm-up ou les learning rate schedules cycliques permettent d'obtenir le bon compromis.

Dans une entreprise de reconnaissance visuelle automobile, le simple fait de doubler la taille du batch tout en réduisant le taux d'apprentissage a permis d'améliorer significativement stabilité et rapidité de convergence sur les modèles embarqués en conditions réelles.

### Stratégie de warm-up

Az Définition

Le **warm-up** consiste à commencer l'entraînement avec un taux d'apprentissage faible, puis à l'augmenter progressivement, afin d'éviter des oscillations initiales trop fortes qui pourraient faire « rater » un bon minimum pour l'optimiseur.

## Débogage des problèmes de régularisation

Une **régularisation mal dosée** bloque l'apprentissage (régularisation trop forte) ou laisse trop de liberté, induisant un surapprentissage (régularisation trop faible). Il faut trouver le bon niveau : l'analyse des courbes (si la validation diverge alors que l'entraînement progresse, c'est souvent une régularisation trop faible) ; les techniques à tester : dropout, early stopping, pénalités L1/L2.

Dans la vision par ordinateur, une équipe française a optimisé un modèle de segmentation d'images en ajustant le ratio de dropout et en introduisant l'arrêt précoce lorsque la validation stagnait.

### Trop de régularisation peut masquer des bugs profonds

⚠ Attention

Une régularisation exagérée « colmate » parfois un vrai problème de données ou d'architecture : si toute modification de la régularisation impacte fortement le résultat, suspectez un problème plus en amont.

## Résolution des problèmes de transfert learning

Le **transfert learning** repose sur la réutilisation de modèles pré-entraînés sur d'autres tâches. Les difficultés fréquentes sont :

- Le **catastrophic forgetting**, où le modèle « oublie » ce qu'il a appris en se spécialisant trop vite sur la nouvelle tâche,
- L'**incompatibilité domaine source/cible**, qui conduit à des performances médiocres voire incohérentes.

Pour diagnostiquer : suivre l'évolution des performances sur la tâche cible ET source après chaque étape, ajuster la quantité de couches gelées ou à fine-tuner, comparer l'apprentissage entre plusieurs modèles pré-entraînés.

Une startup française spécialisée NLP a significativement amélioré ses résultats sur des données médicales francophones en testant des degrés variables de fine-tuning lors de l'adaptation de BERT : la stabilité finale est venue d'une combinaison de gel progressif des couches profondes.

### Documentez chaque étape du fine-tuning

👤 Conseil

Le fine-tuning, surtout quand il génère des changements rapides dans la capacité du modèle, mérite une traçabilité stricte : conserver les checkpoints, métriques intermédiaires, et noter tous les essais pour pouvoir revenir précisément sur chaque scénario.

# V Synthèse

## A. Flashcards

### Pipeline d'IA

Chaîne complète : collecte, préparation, feature engineering, entraînement, validation, déploiement, inférence.

### Débogage stochastique

Erreurs liées à la dimension aléatoire de l'apprentissage, nécessitant analyse de distributions.

### Cycle itératif de débogage

Observation, hypothèses, expérimentation, analyse, itération, documentation systématique.

### Ablation

Retrait progressif de features, couches ou modules pour isoler la cause d'un bug.

### Outlier

Valeur extrême très éloignée de la tendance centrale, signal potentiel d'erreur de saisie.

### Sous-apprentissage

Modèle trop simple ne captant pas la structure, faibles scores en entraînement et validation.

### Neurone mort

Unité retournant toujours zéro, traduisant blocage du gradient ou mauvaise activation.

### Warm-up

Démarrage avec taux d'apprentissage faible puis augmentation progressive pour éviter oscillations.

### Catastrophic forgetting

Oubli des connaissances pré-entraînées lors d'une spécialisation trop rapide en transfert learning.

### Gradient vanishing

Disparition du gradient dans les couches profondes, bloquant l'apprentissage des poids initiaux.

### Gradient explosion

Gradient devenant énorme, entraînant instabilité numérique et divergence de l'entraînement.

### Data leakage

Réinjection accidentelle de la cible ou d'informations futures dans les features d'entraînement.

### Reproductibilité en IA

Capacité à répéter exactement une expérience via seeds, versions et données figées.

### Déséquilibre de classes

Nombre d'exemples très inégal entre classes, biaisant l'apprentissage vers la classe majoritaire.

### Learning rate scheduler

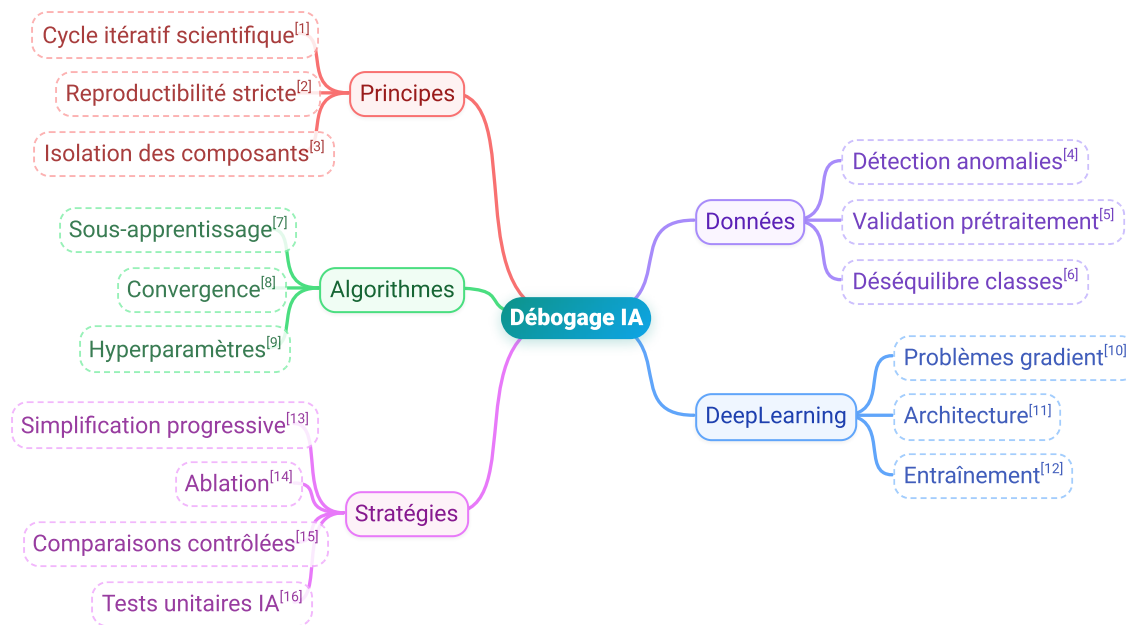
Stratégie d'ajustement adaptatif du taux d'apprentissage au cours de l'entraînement.



## La théorie en Bref

💡 Fondamental

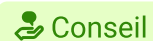
Les **techniques de débogage en IA** exigent une **méthode scientifique rigoureuse** car les erreurs sont souvent **diffuses, stochastiques et non explicites**. Le **cycle itératif** (observation, hypothèses, expérimentation, analyse, documentation) structure le diagnostic en isolant chaque maillon du **pipeline** : données, prétraitement, architecture, hyperparamètres, environnement. La **reproductibilité** (seeds, versions, traçabilité) et l'**isolation des composants** permettent de localiser rapidement la source d'un bug. Les stratégies clés incluent la **simplification progressive**, l'**ablation**, les **comparaisons contrôlées**, les **tests unitaires adaptés** et la **visualisation des gradients et activations**. En Deep Learning, il faut diagnostiquer les problèmes de **gradient** (disparition, explosion), ajuster **batch size et learning rate**, doser la **régularisation** et maîtriser les enjeux du **transfert learning**.



1. Observation, hypothèses, expérimentation, analyse, documentation systématique.
2. Seeds fixés, versions figées, traçabilité complète des expériences.
3. Tester séparément chaque étape du pipeline pour localiser l'erreur.
4. Visualisation distributions, outliers, incohérences, biais de collecte.
5. Comparaison avant/après transformation, tests encodages, data leakage.
6. Analyse répartition, rééchantillonnage, pondération, métriques par classe.
7. Modèle trop simple, manque de features, régularisation excessive.
8. Ajustement learning rate, fonction de perte, initialisation, normalisation.
9. Recherche systématique, optimisation bayésienne, tests incrémentaux.
10. Disparition ou explosion, batch norm, initialisation, visualisation.
11. Complexité adaptée, neurones morts, RNN/Transformers, attention.
12. Batch size, learning rate, régularisation, transfert learning.
13. Réduire complexité étape par étape pour isoler la cause.
14. Retirer features, couches ou modules pour observer l'effet.
15. Comparer versions, jeux de données, groupes pour identifier différences.
16. Valider intégrité de chaque brique du pipeline séparément.

*Débogage IA*

## Les conseils pro



### Comment structurer une investigation de débogage sur un modèle IA en production ?

- Documenter précisément tous les symptômes observés avec dates, contextes et logs d'exécution.
- Formuler plusieurs hypothèses explicites couvrant données, code, paramètres et infrastructure.
- Isoler chaque composant du pipeline pour tester séparément sa contribution au problème.

- Conserver un historique rigoureux de chaque test et modification pour tracer la résolution.

#### **Comment garantir la reproductibilité des expériences de débogage en Deep Learning ?**

- Fixer tous les seeds aléatoires dans le code et les bibliothèques utilisées.
- Versionner strictement les données, le code et les dépendances logicielles.
- Créer un environnement isolé dédié au débogage avec configuration figée.
- Sauvegarder les checkpoints et métriques intermédiaires à chaque étape clé.

#### **Comment diagnostiquer et corriger un problème de gradient dans un réseau profond ?**

- Visualiser les valeurs de gradient couche par couche via TensorBoard ou Weights & Biases.
- Identifier les couches inactives ou instables et analyser leur fonction d'activation.
- Appliquer batch normalization et ajuster l'initialisation des poids selon l'architecture.
- Tester des learning rates adaptatifs et surveiller la stabilité sur plusieurs runs.

#### **Comment valider l'intégrité des données et du prétraitement avant entraînement ?**

- Automatiser des scripts de contrôle de cohérence sur bornes, types et logique métier.
- Comparer visuellement les distributions avant et après chaque transformation appliquée.
- Créer des tests unitaires validant encodages, normalisations et absence de data leakage.
- Conserver un jeu de données brutes non altérées pour audit et comparaison.

#### **Comment optimiser le choix des hyperparamètres lors du débogage d'un modèle ?**

- Ne modifier qu'un seul hyperparamètre par expérience pour isoler son effet précis.
- Utiliser une recherche bayésienne ou grid search systématique sur les paramètres critiques.
- Valider chaque amélioration sur plusieurs jeux de test indépendants pour éviter le surapprentissage.
- Documenter les courbes de sensibilité pour chaque paramètre testé et conserver les configurations optimales.

## VI Auto-évaluation

### Exercice 1 : Spécificités et méthode de débogage IA

#### Exercice

Contrairement au débogage logiciel classique, les erreurs dans un modèle d'IA se manifestent toujours par un message d'erreur explicite ou un arrêt du programme.

Vrai

Faux

#### Exercice

Dans le cycle de débogage itératif en IA, il est recommandé de modifier plusieurs composants simultanément afin d'accélérer la résolution du problème.

Vrai

Faux

#### Exercice

Dans le cycle de débogage itératif décrit dans la leçon, quelle étape suit immédiatement l'expérimentation ?

La documentation systématique des résultats obtenus

L'analyse des écarts et l'interprétation des résultats

La formulation de nouvelles hypothèses

L'observation des symptômes et anomalies

#### Exercice

Une équipe d'assurance a réussi à reproduire un bug lié au passage à l'heure d'hiver grâce à son environnement de débogage. Quelle pratique a rendu cela possible ?

L'utilisation d'un modèle de référence (baseline) plus simple

L'application de la simplification progressive du pipeline

La fixation des seeds aléatoires, le gel des versions logicielles et la documentation de l'état des données

L'analyse différentielle entre deux versions du modèle

## Exercice 6 : Débogage des données et du prétraitement

### Exercice

Lors de l'analyse d'un modèle de crédit scoring dont les performances ont chuté, quelles actions font partie d'un diagnostic rigoureux des données d'entrée ?

- ☐ **A** Visualiser les distributions des variables à l'aide d'histogrammes et de boxplots
- ☐ **B** Augmenter immédiatement la complexité du modèle pour compenser les anomalies
- ☐ **C** Détecter les outliers via l'écart interquartile ou la méthode de Mahalanobis
- ☐ **D** Vérifier la cohérence des encodages entre les différentes sources de données
- ☐ **E** Réduire le taux d'apprentissage pour stabiliser les prédictions

### Exercice

Un modèle de détection de fraude bancaire prédit quasi-systématiquement 'non fraude', même sur des cas réels de fraude. Quelle est la cause la plus probable de ce comportement ?

- ☐ Un taux d'apprentissage trop élevé qui fait diverger la fonction de perte
- ☐ Un déséquilibre de classes : les exemples de fraude sont bien moins nombreux que les cas normaux
- ☐ Une architecture réseau trop simple qui ne capte pas la complexité du problème
- ☐ Un problème de gradient vanishing dans les couches profondes du modèle

### Exercice

Parmi les situations suivantes, lesquelles illustrent des bugs 'silencieux' typiques du prétraitement, tels que décrits dans la leçon ?

- ☐ **A** Une normalisation appliquée deux fois sur les mêmes données
- ☐ **B** Un crash du pipeline lors de la lecture d'un fichier CSV corrompu
- ☐ **C** Une inversion accidentelle des canaux couleur d'images en entrée du modèle
- ☐ **D** Un message d'erreur explicite signalant une valeur manquante dans une colonne



Un data leakage causé par la réinjection accidentelle de la variable cible dans les features

## Exercice 10 : Simplification progressive et ablation

### Exercice

Associez chaque stratégie de débogage à sa description correcte.

**A** Simplification progressive

**B** Ablation

**C** Débogage par contrastes

**D** Test unitaire IA

|                                                                                                                        |                                                                                                                       |                                                                                                                          |                                                                                                         |
|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Retrait successif de features, couches ou modules pour observer leur effet sur la performance ou l'apparition d'un bug | Réduction étape par étape de la complexité du modèle ou du pipeline pour identifier le point de bascule d'un problème | Comparaison systématique des cas où le modèle fonctionne versus ceux où il échoue pour isoler les facteurs discriminants | Vérification de l'intégrité, de la cohérence et de la stabilité de chaque brique du pipeline séparément |
|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|

### Exercice

Lors d'un débogage par ablation, vous retirez successivement des couches d'un réseau et constatez que le bug de surapprentissage disparaît à partir d'un certain niveau de profondeur. Quelle conclusion pouvez-vous en tirer ?

Le modèle souffre d'un problème de gradient vanishing dans les couches retirées

Le problème est lié à la gestion d'un composant spécifique qui n'intervient qu'à partir d'une certaine profondeur

Les données d'entraînement contiennent des outliers qui perturbent les couches profondes

Le taux d'apprentissage est inadapté à la profondeur du réseau

## Exercice 13 : Débogage des réseaux de neurones profonds

### Exercice

Associez chaque problème de gradient à ses caractéristiques et conséquences observables.

**A** Neurone mort

**B** Gradient explosion

**C** Surapprentissage architectural

**D** Gradient vanishing

|                                                                                                   |                                                                                                                 |                                                                                                                                  |                                                                                                                    |
|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Le gradient devient quasi nul dans les couches profondes, bloquant l'apprentissage de ces couches | Le gradient prend des valeurs extrêmes, provoquant une instabilité numérique et la divergence de l'entraînement | Une unité retourne toujours la même valeur quelle que soit l'entrée, souvent à cause d'un mauvais choix de fonction d'activation | Le réseau mémorise le bruit d'entraînement et décroche sur de nouvelles données en raison d'un excès de paramètres |
|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|

## Exercice

Remettez dans le bon ordre les étapes d'un diagnostic de problème d'attention dans un modèle Transformer, tel que décrit dans la leçon.

**A** Ajuster l'architecture ou les paramètres d'entraînement et valider sur les séquences tests

**B** Visualiser les poids d'attention et analyser leur distribution couche par couche

**C** Identifier les tokens qui monopolisent indûment l'attention ou les dépendances non capturées

**D** Constituer des jeux de séquences tests variés, incluant des cas extrêmes (phrases longues, exemples rares)

**E** Monitorer l'évolution des gradients à travers le temps pour confirmer l'hypothèse

Réponse : \_\_\_\_ \_

## Exercice

Une startup NLP adapte un modèle BERT pré-entraîné à des données médicales francophones. Après quelques époques de fine-tuning intensif, les performances sur la tâche cible s'améliorent mais celles sur la tâche d'origine s'effondrent. De quel phénomène s'agit-il ?

Gradient vanishing dû à la profondeur du modèle BERT

Catastrophic forgetting : le modèle oublie ses représentations initiales en se spécialisant trop vite

Déséquilibre de classes dans le jeu de données médical

Surapprentissage causé par un taux de dropout trop faible



## VII Exercice : Déboguer un modèle de reconnaissance d'images en production

### Mise en situation

Vous intégrez l'équipe IA d'**ImageStyle**, une entreprise française qui développe des solutions de reconnaissance d'images pour le secteur de la mode. L'entreprise commercialise une API de classification automatique de vêtements, utilisée par plusieurs e-commerçants pour taguer automatiquement leurs catalogues produits.

Le modèle en production repose sur un **réseau de neurones convolutif (CNN)** entraîné sur un dataset de **250 000 images** réparties en **35 catégories** de vêtements (robes, pantalons, vestes, accessoires, chaussures, etc.). Le modèle a été fine-tuné à partir d'un backbone **ResNet-50** pré-entraîné sur ImageNet.

### Symptômes observés

Depuis le déploiement de la dernière version (v2.3), trois problèmes majeurs ont été remontés par l'équipe produit et les clients :

- **Symptôme 1 — Confusions récurrentes entre catégories** : le modèle confond systématiquement les "blazers" avec les "vestes en jean", et les "jupes longues" avec les "robes". Le taux de précision sur ces catégories est passé de **89%** à **62%** entre la v2.2 et la v2.3.
- **Symptôme 2 — Dégradation sur les nouvelles collections** : les performances globales chutent de **7 points de F1-score** lorsque le modèle traite les images de la collection printemps-été 2025, alors qu'il reste stable sur les collections précédentes. L'équipe suspecte un **data shift saisonnier**.
- **Symptôme 3 — Temps d'inférence anormalement long** : le temps moyen d'inférence est passé de **45 ms** à **120 ms** par image après la mise à jour, sans modification apparente de l'infrastructure serveur (GPU NVIDIA A100, même configuration).

### Données complémentaires

Voici les informations dont vous disposez pour mener votre investigation :

| Élément                      | Détail                                                                                                                                               |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Framework                    | PyTorch 2.1                                                                                                                                          |
| Backbone                     | ResNet-50 fine-tuné                                                                                                                                  |
| Dataset d'entraînement       | 250 000 images, 35 catégories                                                                                                                        |
| Dernière modification (v2.3) | Ajout de 5 nouvelles catégories, passage du batch size de 64 à 128, mise à jour de la librairie de prétraitement d'images (Albumentations 1.3 → 1.4) |
| Outils de suivi              | MLflow pour le tracking des expériences, TensorBoard pour la visualisation                                                                           |
| Jeu de test fixe             | 15 000 images annotées manuellement, non modifié depuis la v2.0                                                                                      |
| Métriques disponibles        | Précision, rappel, F1-score par catégorie, matrice de confusion, courbes de loss                                                                     |

L'équipe précédente n'a pas documenté les raisons précises du changement de batch size ni vérifié l'impact de la mise à jour d'Albumentations sur le pipeline de prétraitement.

### **Question 1**

Concevez une démarche structurée de débogage pour ce modèle. Décrivez les grandes étapes de votre investigation, en précisant comment vous appliquez le cycle itératif (observation, hypothèses, expérimentation, analyse) à la situation d'ImageStyle.

### **Question 2**

Proposez des expériences précises à mener pour tester différentes hypothèses sur l'origine de chacun des trois symptômes. Pour chaque symptôme, formulez au moins une hypothèse et décrivez l'expérience contrôlée correspondante.

### **Question 3**

Suggérez des techniques de débogage (analyse, outils ou méthodes) adaptées à chaque symptôme observé. Justifiez vos choix en vous appuyant sur les spécificités du contexte d'ImageStyle.

### **Question 4**

Élaborez un plan d'action prioritaire : ordonnez vos tâches et interventions de débogage, en précisant pourquoi vous priorisez ainsi. Tenez compte de l'impact métier de chaque symptôme pour ImageStyle.